

So far, one problem still persists...

- ◆ *Busy waiting!*
- ◆ Alternative approach
 - ◆ If thread/process unable to enter critical region
 - ◆ Thread/process can be put to *sleep* (i.e., it gets *blocked*)
 - ◆ *Wake up* signal sent to thread/process when it becomes eligible to enter critical section

One solution...

- ◆ Shared variable *mutex* that can be in one of two states
 - ◆ *Locked* or *1*
 - ◆ *Unlocked* or *0*
- ◆ *Can be used to implement software lock without busy waiting*

Mutex operation

```
mutex_lock:
    TSL REGISTER,MUTEX      | copy mutex to register and set mutex to 1
    CMP REGISTER,#0         | was mutex zero?
    JZE ok                  | if it was zero, mutex was unlocked, so return
    CALL thread_yield        | mutex is busy; schedule another thread
    JMP mutex_lock           | try again
ok:
    RET                     | return to caller; critical region entered
```

```
mutex_unlock:
    MOVE MUTEX,#0           | store a 0 in mutex
    RET                     | return to caller
```

- ◆ Similar to software lock with *enter_region* & *leave_region*
- ◆ Crucial difference
 - ◆ No continuous checking of mutex, i.e., no *busy waiting*